



1-5 September 2025
Valencia, Spain



Electrical &
Computer
Engineering



Computer Science
UNIVERSITY OF TORONTO

li.u LINKÖPING
UNIVERSITY

LLM-based Satisfiability Checking of String Requirements by Consistent Data and Checker Generation

Boqi Chen^{*,1}, Aren A. Babikian^{*,2}, **Shuzhao Feng**¹,
Dániel Varró^{1,3}, Gunter Mussbacher¹

4 September, 2025

* Both authors contributed equally to the research

¹ Electrical and Computer Engineering, McGill University, Montreal, Canada

² Department of Computer Science, University of Toronto, Toronto, Canada

³ Department of Computer and Information Science (IDA), Linköping University, Linköping, Sweden



Example of Requirements Verification

- R1.** The system shall require an email address to contain exactly one “@” character.
- R2.** The system shall require an email address not to end with a “.” character.
- R3.** The system shall require an email to contain at least one “.” character after the “@” character.

Example of Requirements Verification

- R1.** The system shall require an email address to contain exactly one “@” character.
- R2.** The system shall require an email address not to end with a “.” character.
- R3.** The system shall require an email to contain at least one “.” character after the “@” character.

Requirement Set	Behavior	Satisfiable?	Example Data
$R1 \wedge R2 \wedge R3$	Store email	?	?
$\neg R1 \wedge R2 \wedge R3$	Show error	?	?
$R1 \wedge \neg R2 \wedge \neg R3$	Show error	?	?

Example of Requirements Verification

- R1.** The system shall require an email address to contain exactly one “@” character.
- R2.** The system shall require an email address not to end with a “.” character.
- R3.** The system shall require an email to contain at least one “.” character after the “@” character.

Requirement Set	Behavior	Satisfiable?	Example Data
$R1 \wedge R2 \wedge R3$	Store email	Yes	?
$\neg R1 \wedge R2 \wedge R3$	Show error	Yes	?
$R1 \wedge \neg R2 \wedge \neg R3$	Show error	No	?

Example of Requirements Verification

- R1.** The system shall require an email address to contain exactly one “@” character.
- R2.** The system shall require an email address not to end with a “.” character.
- R3.** The system shall require an email to contain at least one “.” character after the “@” character.

Requirement Set	Behavior	Satisfiable?	Example Data
$R1 \wedge R2 \wedge R3$	Store email	Yes	“ab@cd.com”
$\neg R1 \wedge R2 \wedge R3$	Show error	Yes	“ab@@cd.com”
$R1 \wedge \neg R2 \wedge \neg R3$	Show error	No	-

Existing Methods for Satisfiability Checking

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers
 - Time consuming, complexity increases with the number of requirements

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers
 - Time consuming, complexity increases with the number of requirements
- Formal constraint solvers, e.g. satisfiability modulo theories (SMT) solvers

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers
 - Time consuming, complexity increases with the number of requirements
- Formal constraint solvers, e.g. satisfiability modulo theories (SMT) solvers
 - Requires formal syntax

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers
 - Time consuming, complexity increases with the number of requirements
 - Formal constraint solvers, e.g. satisfiability modulo theories (SMT) solvers
 - Requires formal syntax
- Natural language requirement.**
The system shall require an email address to contain exactly one “@” character.

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers
 - Time consuming, complexity increases with the number of requirements
- Formal constraint solvers, e.g. satisfiability modulo theories (SMT) solvers
 - Requires formal syntax

Natural language requirement.

The system shall require an email address to contain exactly one “@” character.

Declarative checker.

```
(and (str.contains s "@") (not (str.in.re s (re.++ (re.* re.allchar)
(str.to.re "@" (re.* re.allchar) (str.to.re "@" (re.* re.allchar)))))
```

Existing Methods for Satisfiability Checking

- Manually done by requirement engineers
 - Time consuming, complexity increases with the number of requirements
- Formal constraint solvers, e.g. satisfiability modulo theories (SMT) solvers
 - Requires formal syntax

Natural language requirement.
The system shall require an email address to contain exactly one “@” character.

Declarative checker.

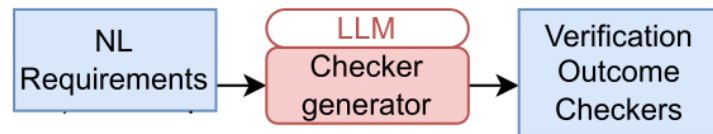
```
(and (str.contains s "@") (not (str.in.re s (re.++ (re.* re.allchar) (str.to.re "@" (re.* re.allchar) (str.to.re "@" (re.* re.allchar))))))
```
 - Formal syntax often unavailable and has to be done by (experienced) requirement engineers.

Our Proposition

1. Using large language models (LLMs):

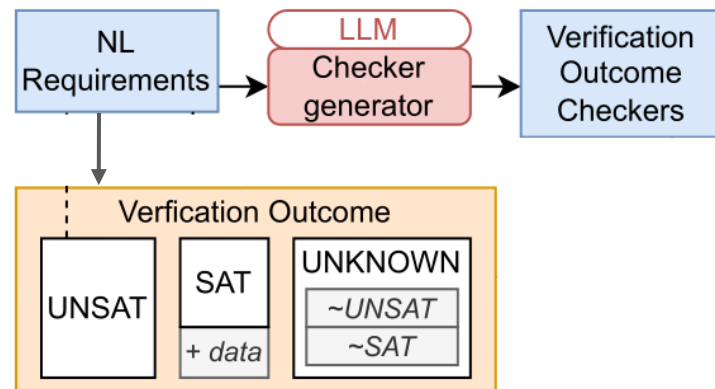
Our Proposition

1. Using large language models (LLMs):
 - a. Generate Requirement Verification Checkers,



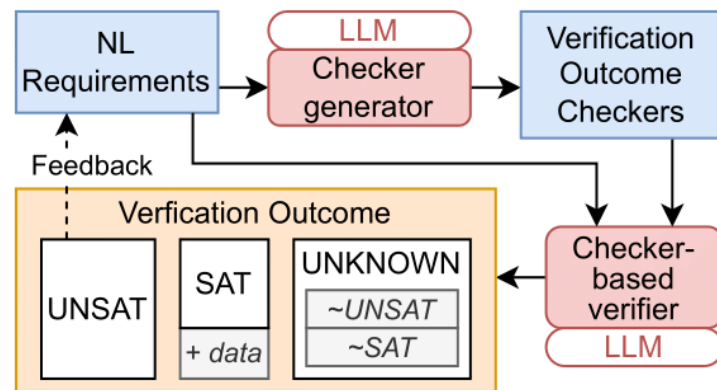
Our Proposition

1. Using large language models (LLMs):
 - a. Generate Requirement Verification Checkers,
 - b. Generate the LLM-based Verification Outcome (LVO),

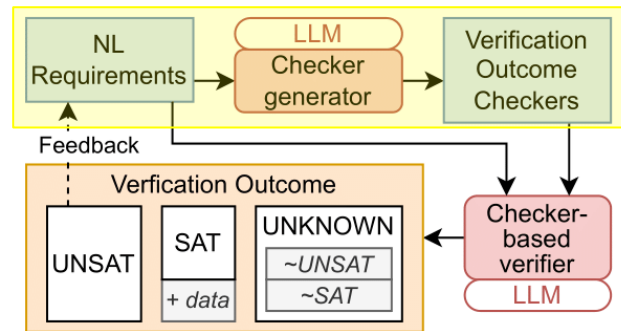


Our Proposition

1. Using large language models (LLMs):
 - a. Generate Requirement Verification Checkers,
 - b. Generate the LLM-based Verification Outcome (LVO),
2. Use checkers to verify the soundness of the LVO and retry if necessary.



Checker Generation Overview



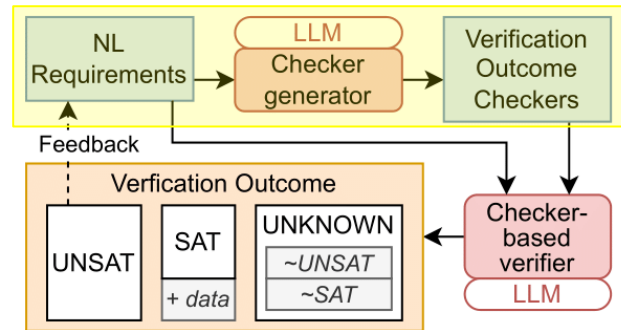
Checker Generation Overview

Individual Generation

Each requirement is its own batch, generate individually.

Batch Generation

Group several requirements together and generate them at once.



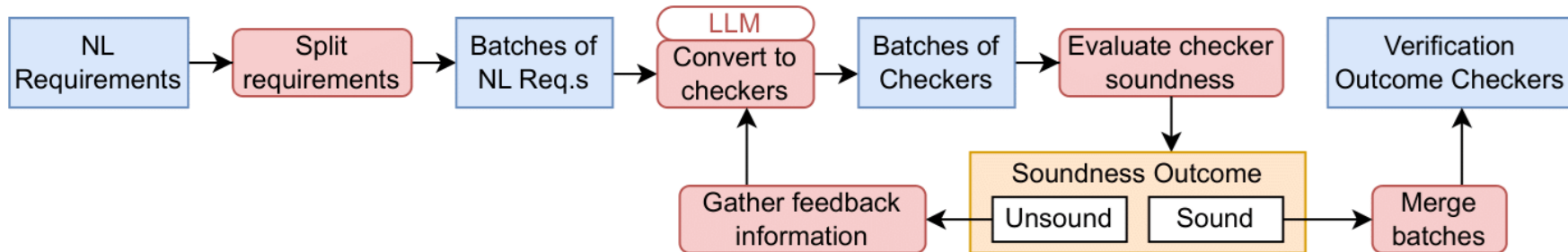
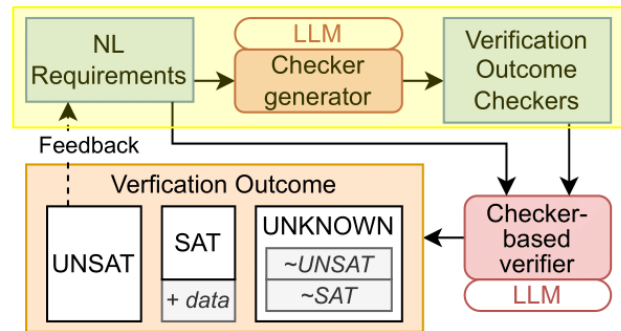
Checker Generation Overview

Individual Generation

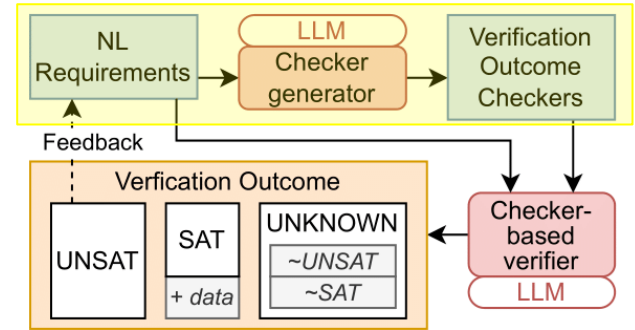
Each requirement is its own batch, generate individually.

Batch Generation

Group several requirements together and generate them at once.



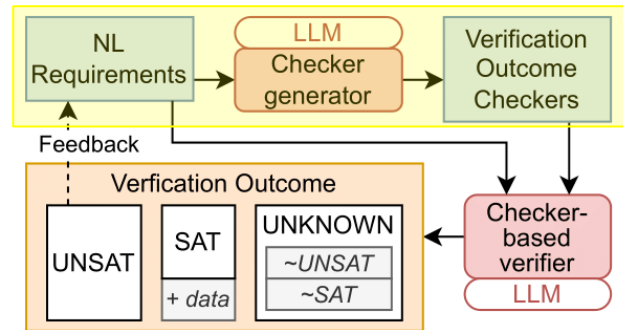
Declarative vs Imperative Checkers



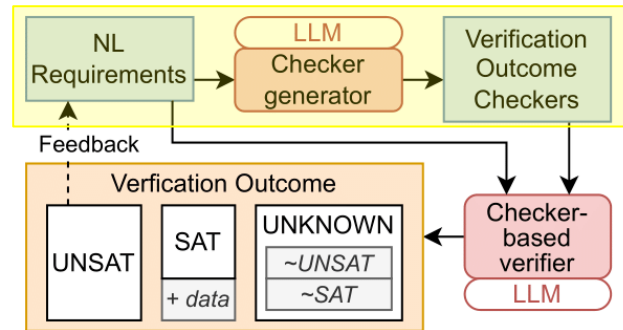
Declarative vs Imperative Checkers

Natural language requirement.

The system shall require an email address to contain exactly one “@” character.



Declarative vs Imperative Checkers



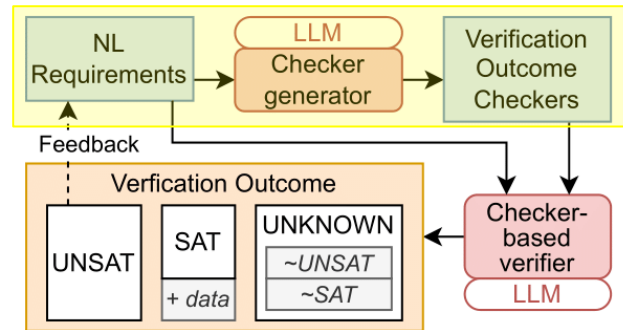
Natural language requirement.

The system shall require an email address to contain exactly one “@” character.

Declarative checker.

```
(and (str.contains s "@") (not (str.in.re s (re.++ (re.*
re.allchar) (str.to.re "@") (re.* re.allchar) (str.to.re "@")
(re.* re.allchar))))))
```

Declarative vs Imperative Checkers



Natural language requirement.

The system shall require an email address to contain exactly one “@” character.

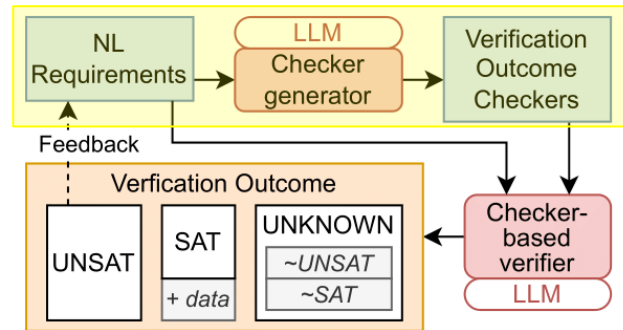
Declarative checker.

Data Check

Satisfiability Check

```
(and (str.contains s "@") (not (str.in.re s (re.++ (re.*
re.allchar) (str.to.re "@") (re.* re.allchar) (str.to.re "@")
(re.* re.allchar)))))
```

Declarative vs Imperative Checkers



Natural language requirement.

The system shall require an email address to contain exactly one “@” character.

Declarative checker.

Data Check

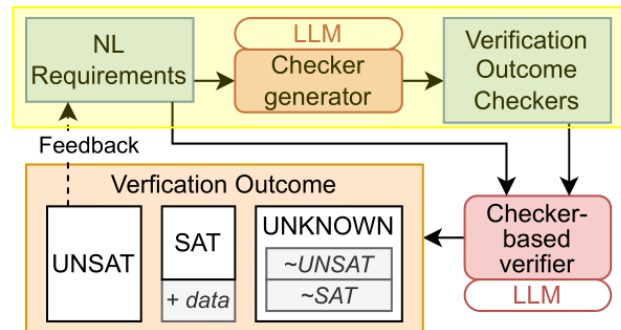
Satisfiability Check

```
(and (str.contains s "@") (not (str.in.re s (re.++ (re.* re.allchar) (str.to.re "@") (re.* re.allchar))))))
```

Imperative checker.

```
def this_function(s: str) -> bool:
    return s.count("@") == 1
```

Declarative vs Imperative Checkers



Natural language requirement.

The system shall require an email address to contain exactly one “@” character.

Declarative checker.

Data Check

Satisfiability Check

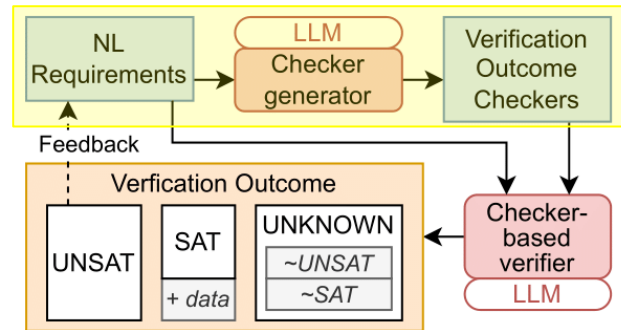
```
(and (str.contains s "@") (not (str.in.re s (re.++ (re.* re.allchar) (str.to.re "@") (re.* re.allchar))))))
```

Imperative checker.

Data Check

```
def this_function(s: str) -> bool:
    return s.count("@") == 1
```

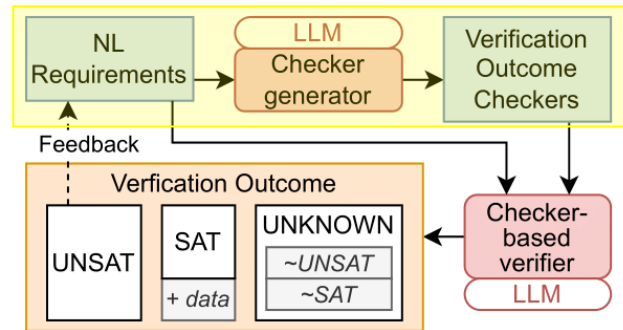
Checker Soundness Metrics



Checker Soundness Metrics

Syntax Accuracy

Proportion of generated checkers without SMT/Python syntax errors.



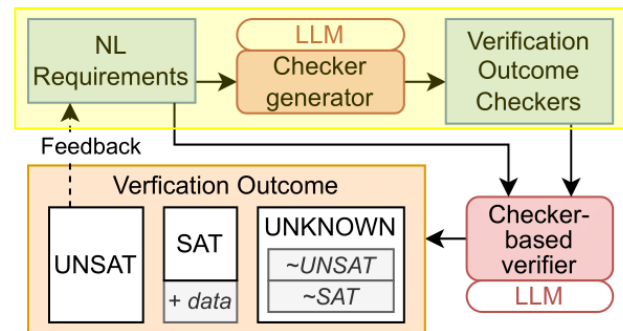
Checker Soundness Metrics

Syntax Accuracy

Proportion of generated checkers without SMT/Python syntax errors.

Testing Accuracy

Proportion of generated checkers passing all of the predefined unit tests.



Checker Soundness Metrics

Syntax Accuracy

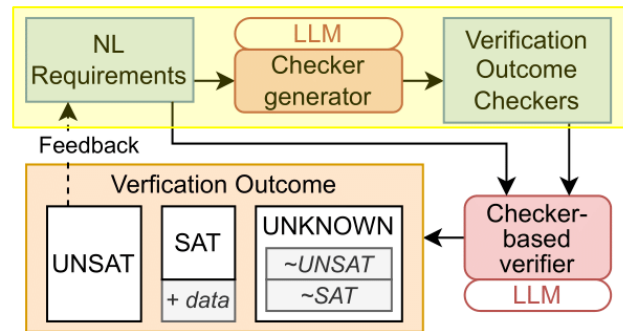
Proportion of generated checkers without SMT/Python syntax errors.

Testing Accuracy

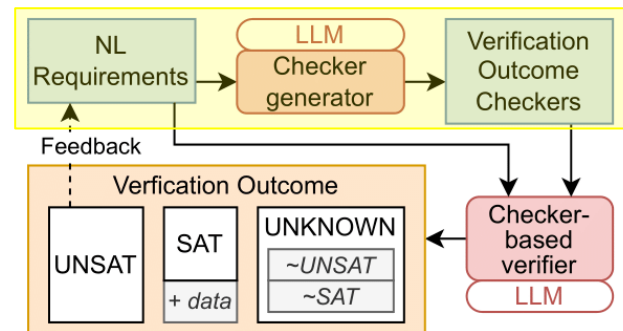
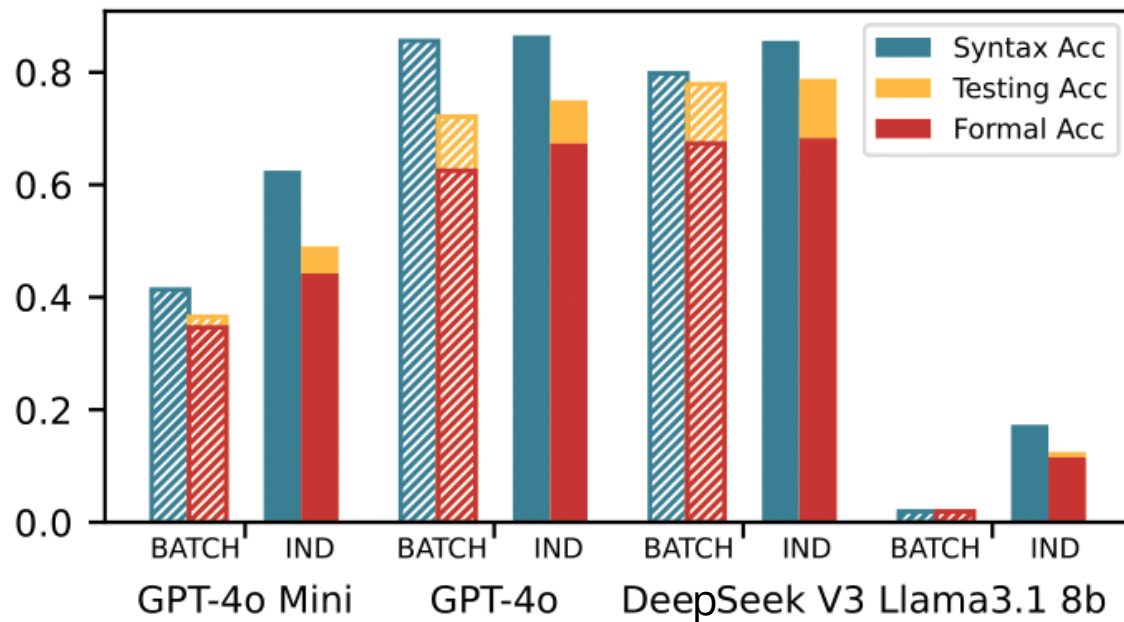
Proportion of generated checkers passing all of the predefined unit tests.

Formal Accuracy (declarative checkers only)

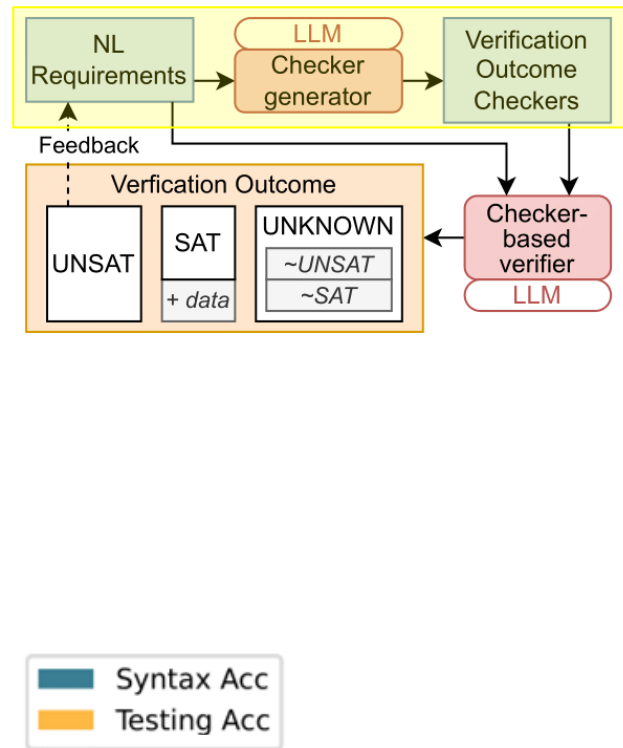
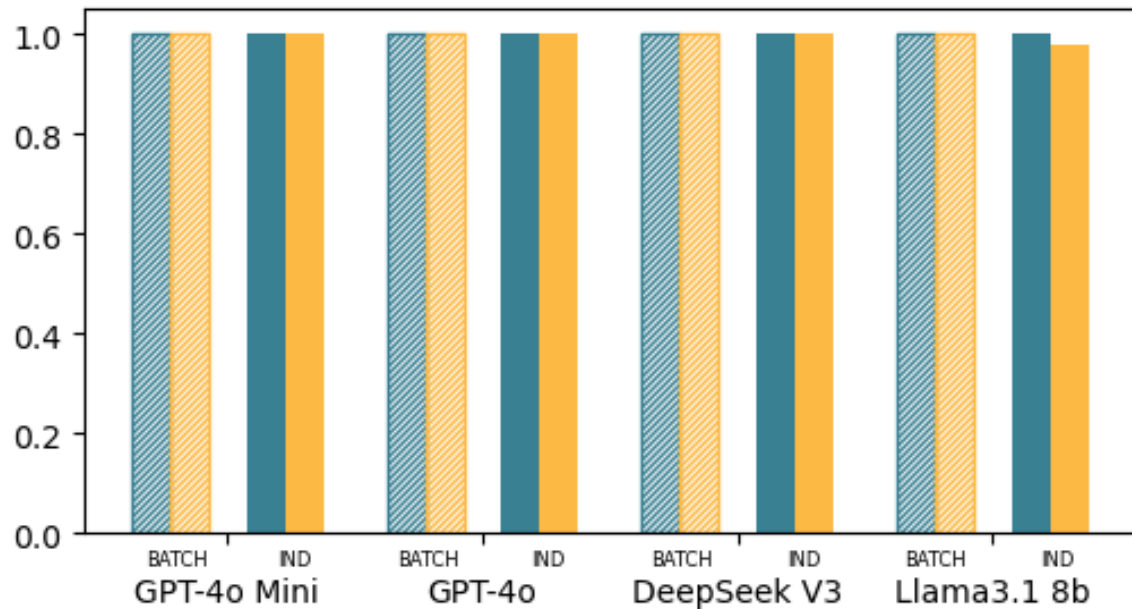
Proportion of generated checkers formally equivalent to the ground truth.



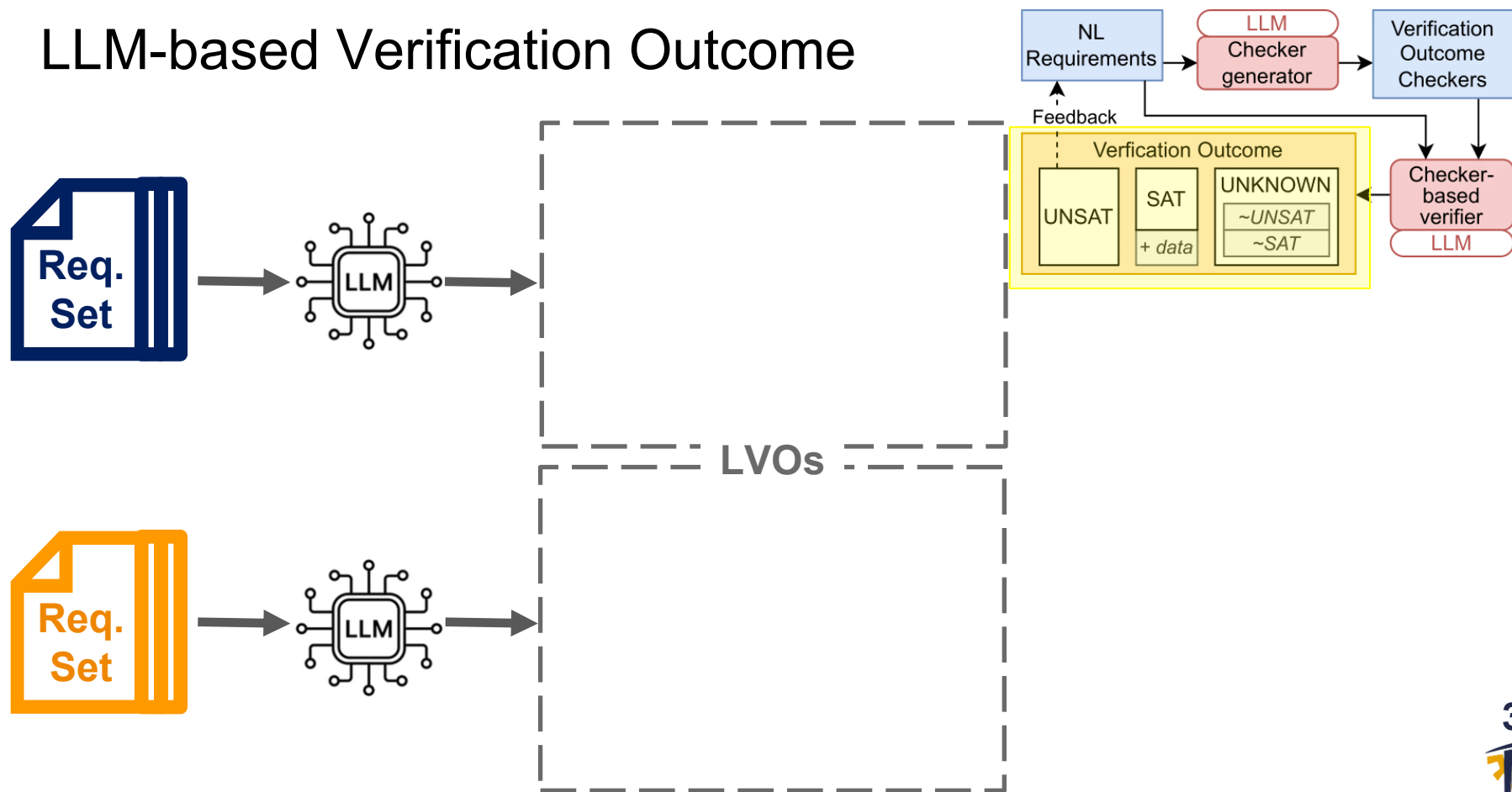
Soundness of Declarative (SMT) Checkers



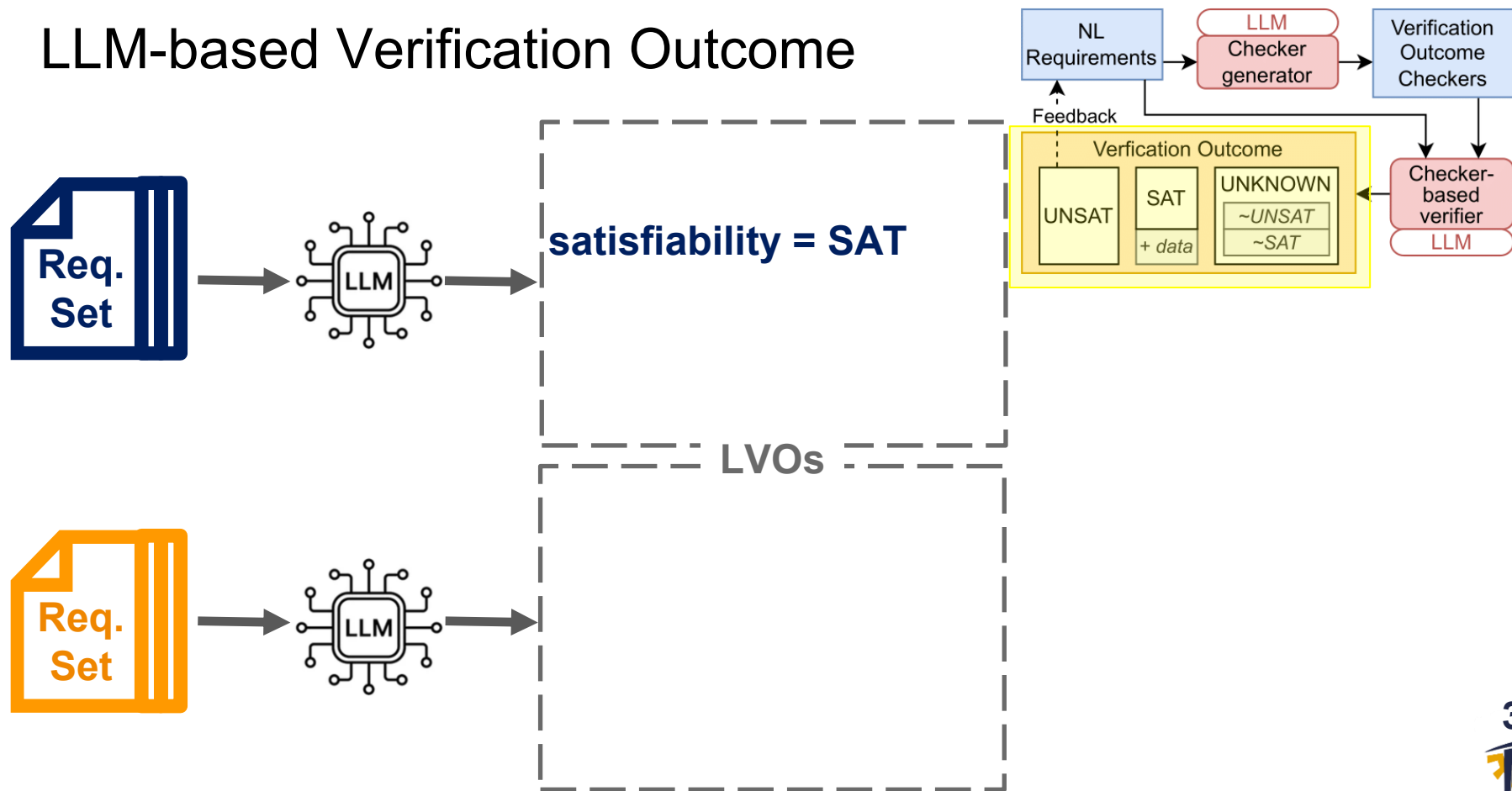
Soundness of Imperative (Python) Checkers



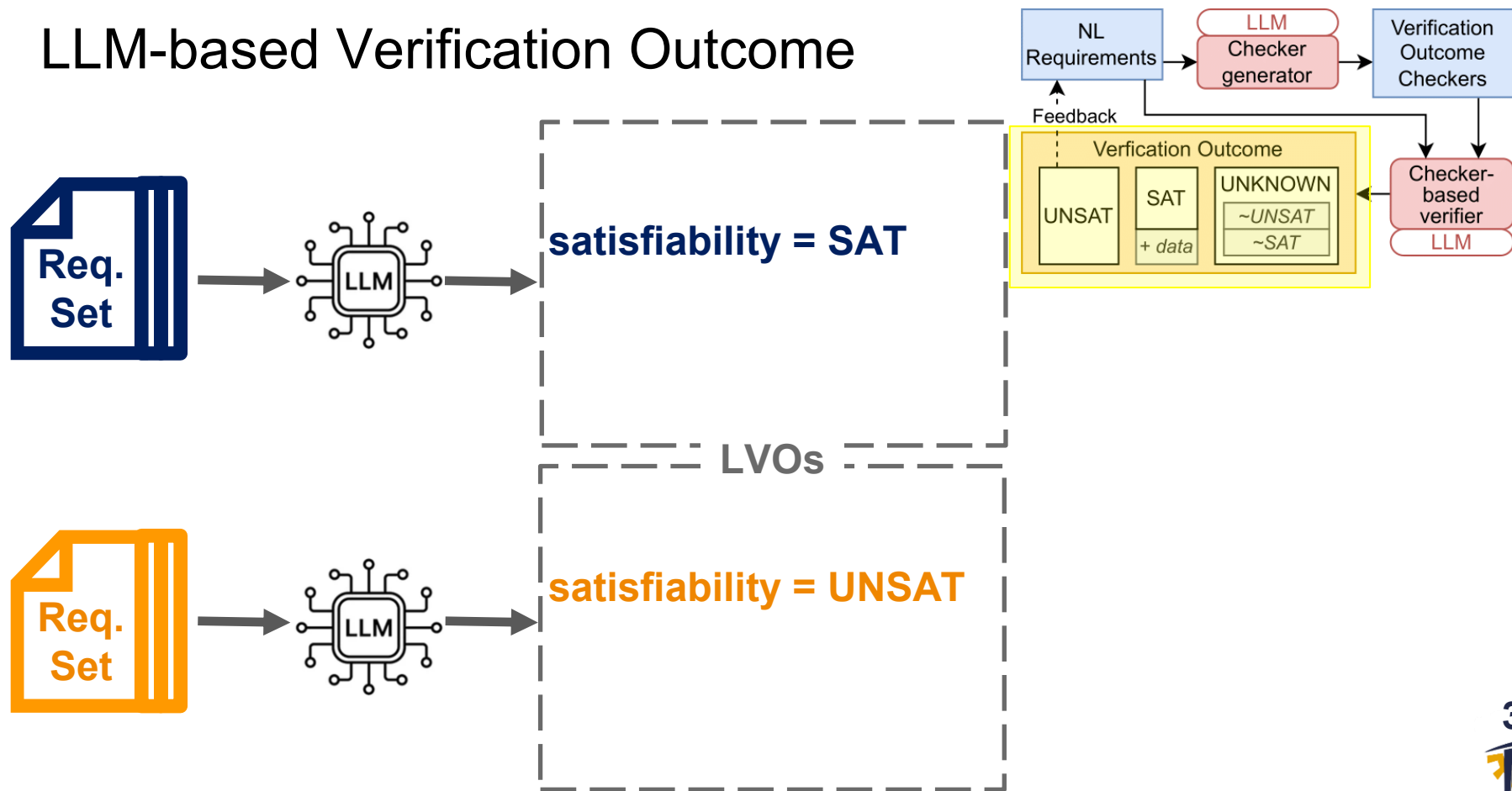
LLM-based Verification Outcome



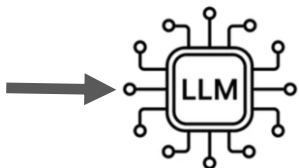
LLM-based Verification Outcome



LLM-based Verification Outcome

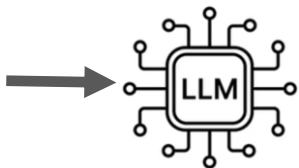


LLM-based Verification Outcome

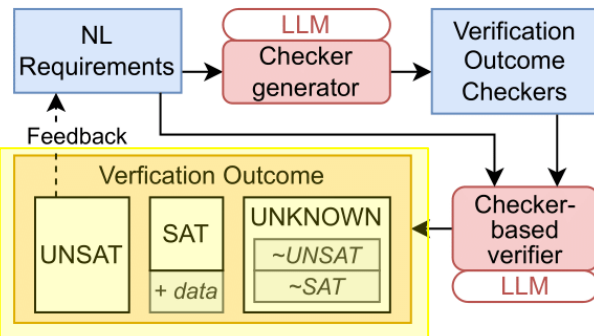


satisfiability = SAT
data = some@mail.com

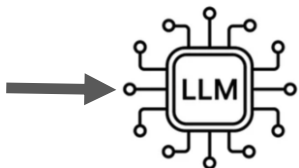
LVOs



satisfiability = UNSAT

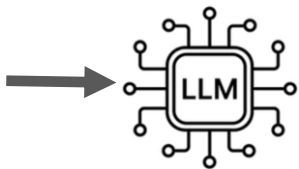


LLM-based Verification Outcome

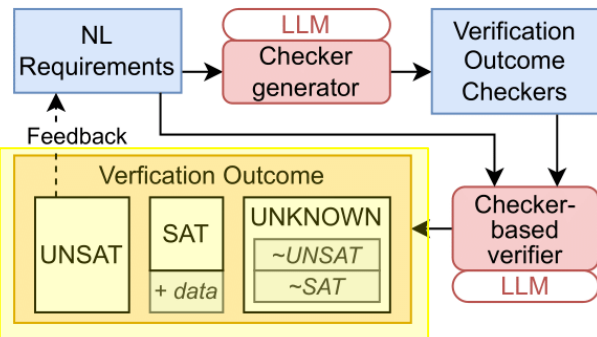


satisfiability = SAT
data = some@mail.com

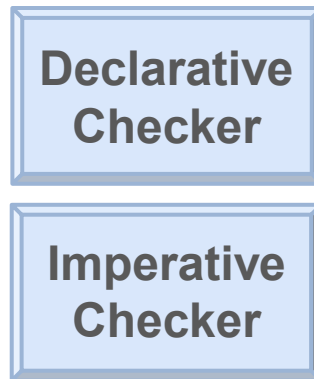
LVOs



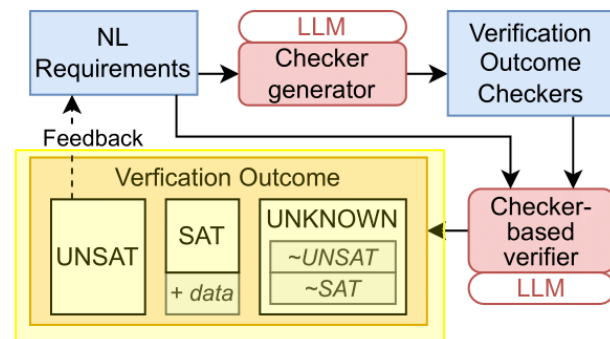
satisfiability = UNSAT
data = (N/A)



Checker-based LVO Verification



correct
or
incorrect



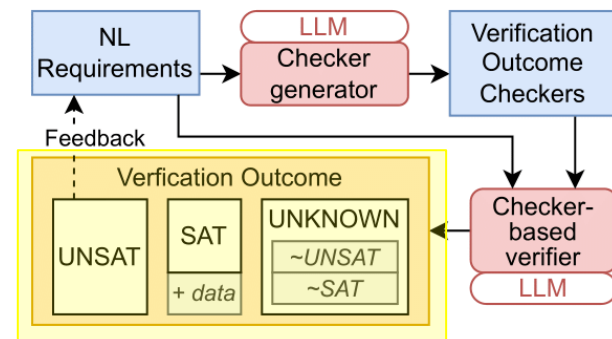
Checker-based LVO Verification Example 1

some@@mail.com

The system shall require an email address to contain exactly one "@"

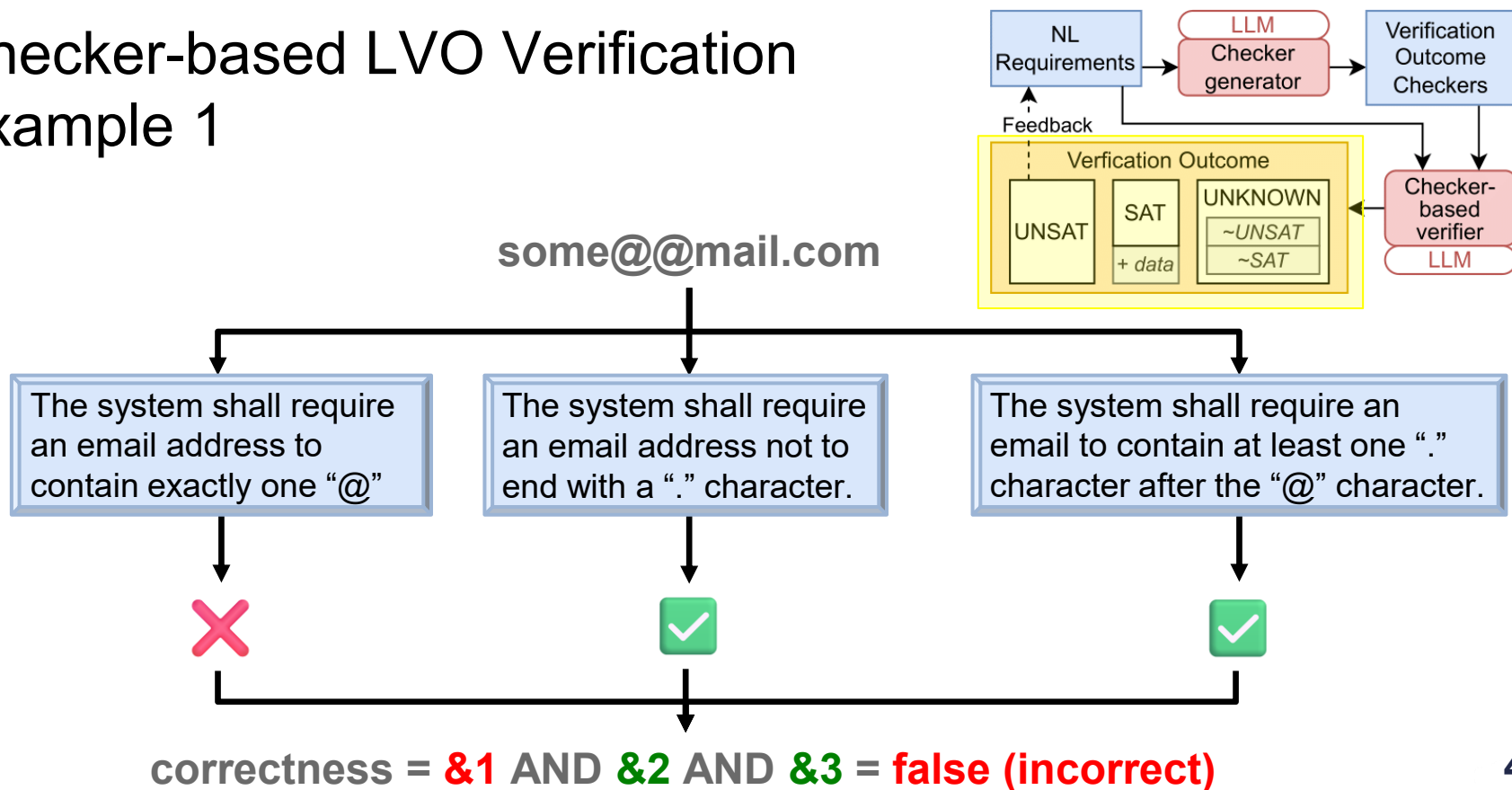
The system shall require an email address not to end with a "." character.

The system shall require an email to contain at least one "." character after the "@" character.

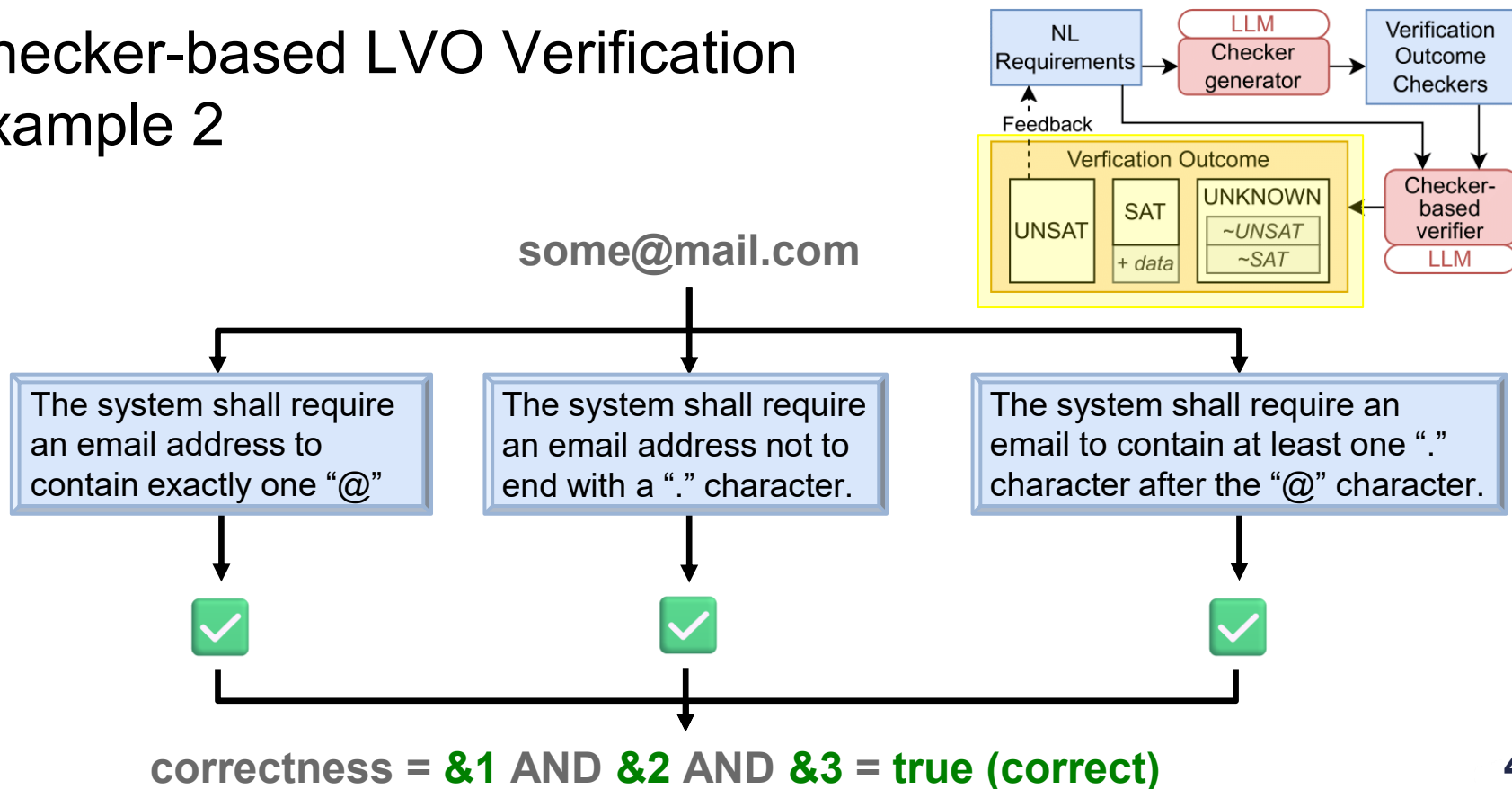


Checker-based LVO Verification

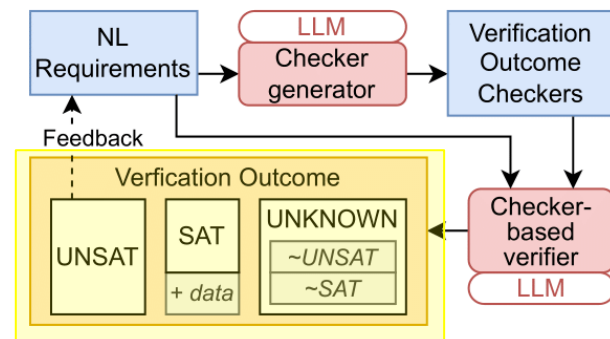
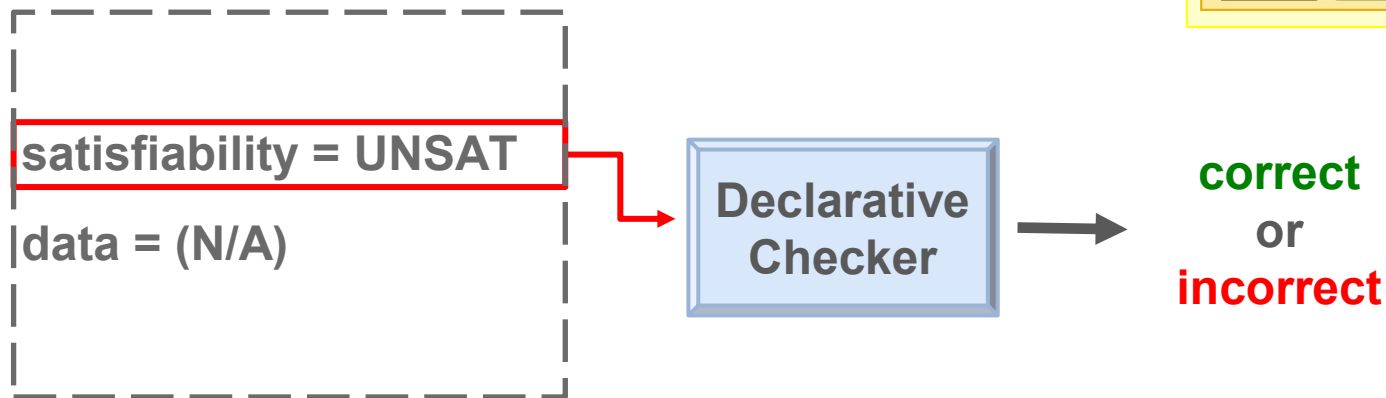
Example 1



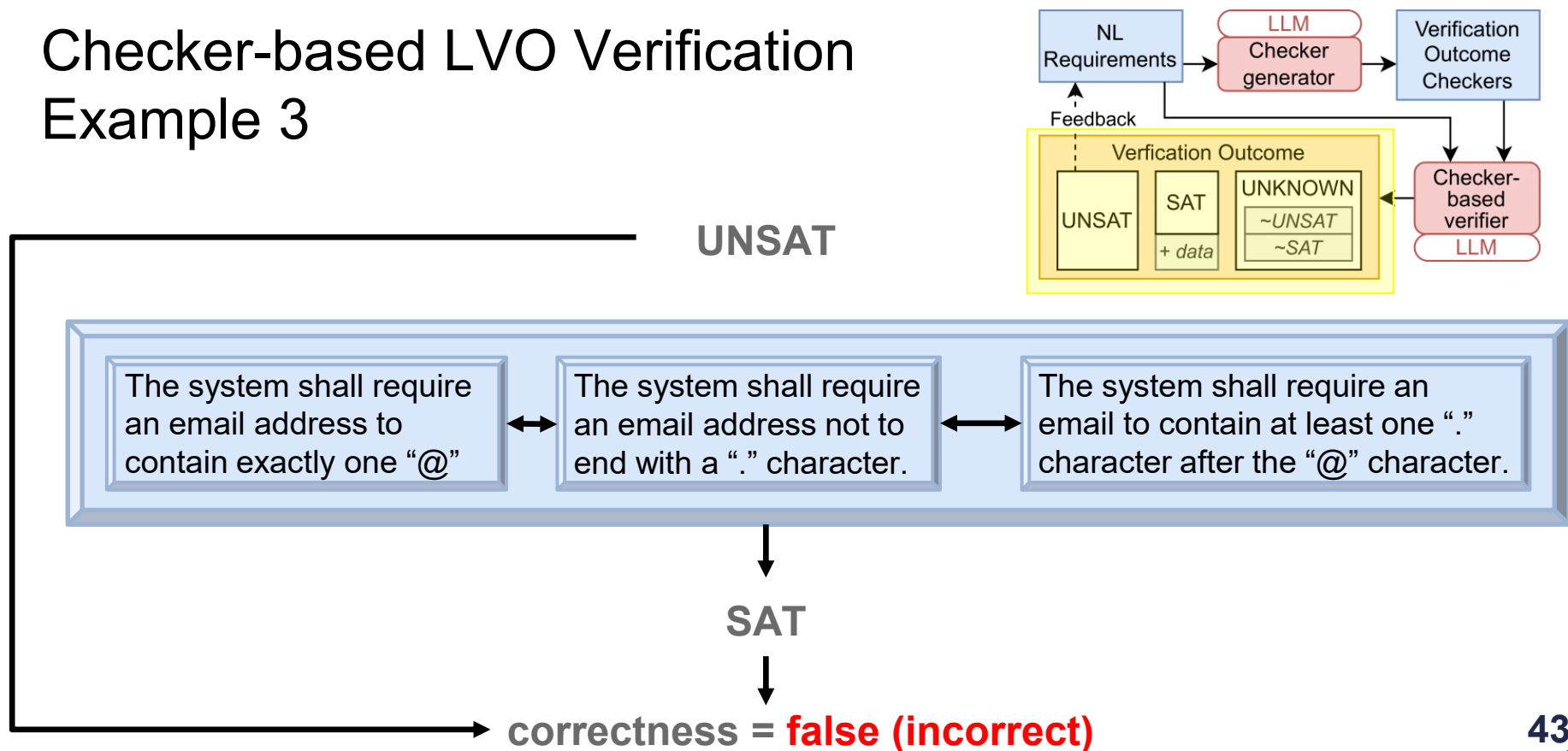
Checker-based LVO Verification Example 2



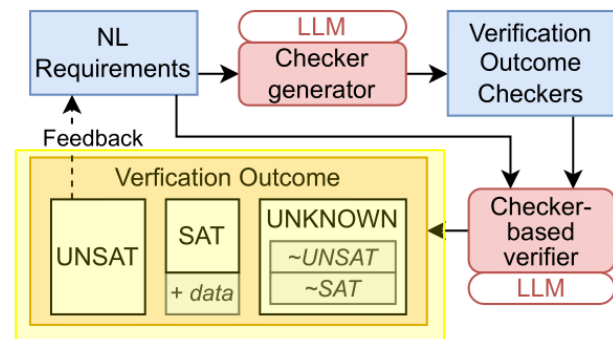
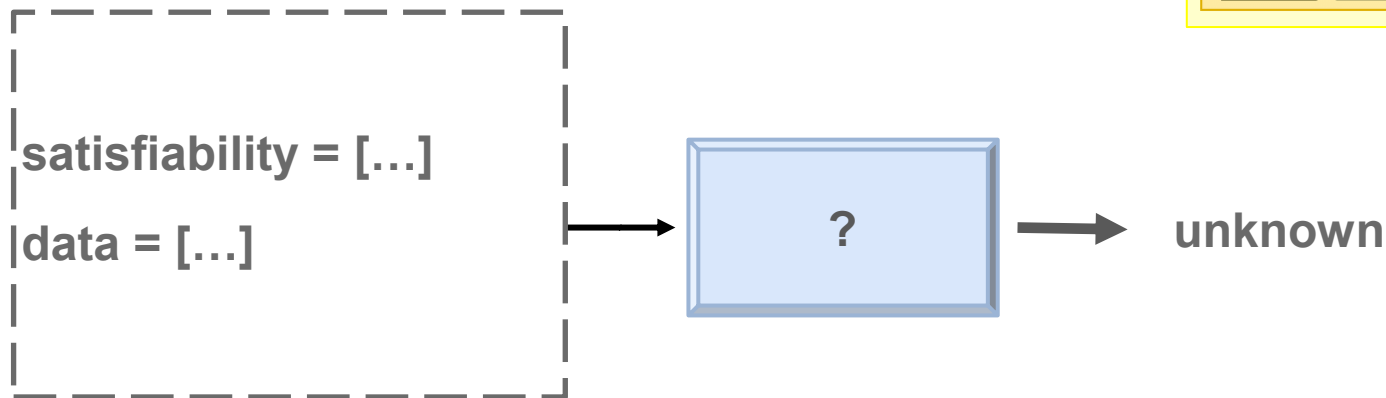
Checker-based LVO Verification



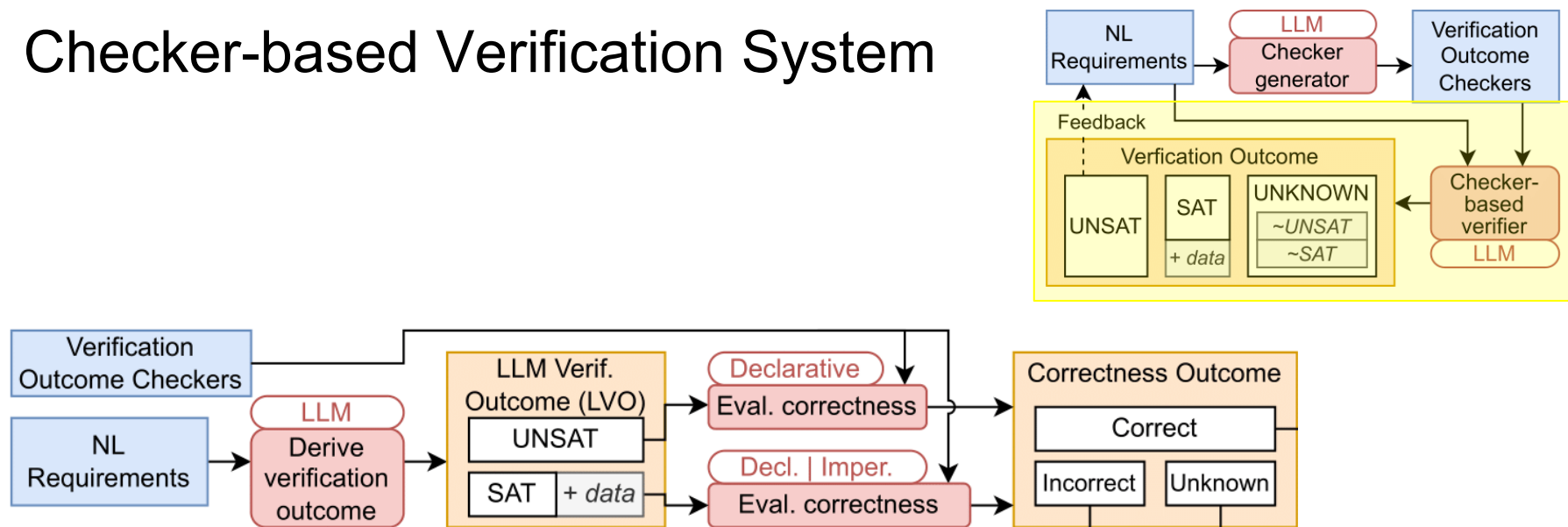
Checker-based LVO Verification Example 3



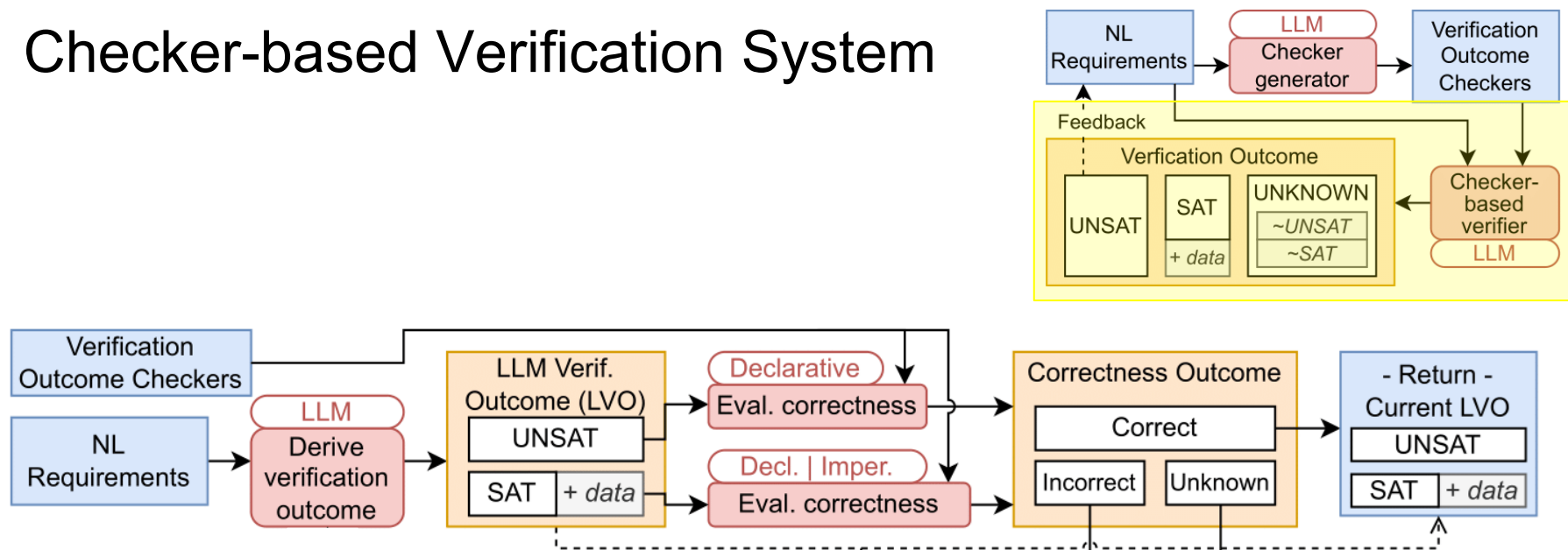
Checker-based LVO Verification



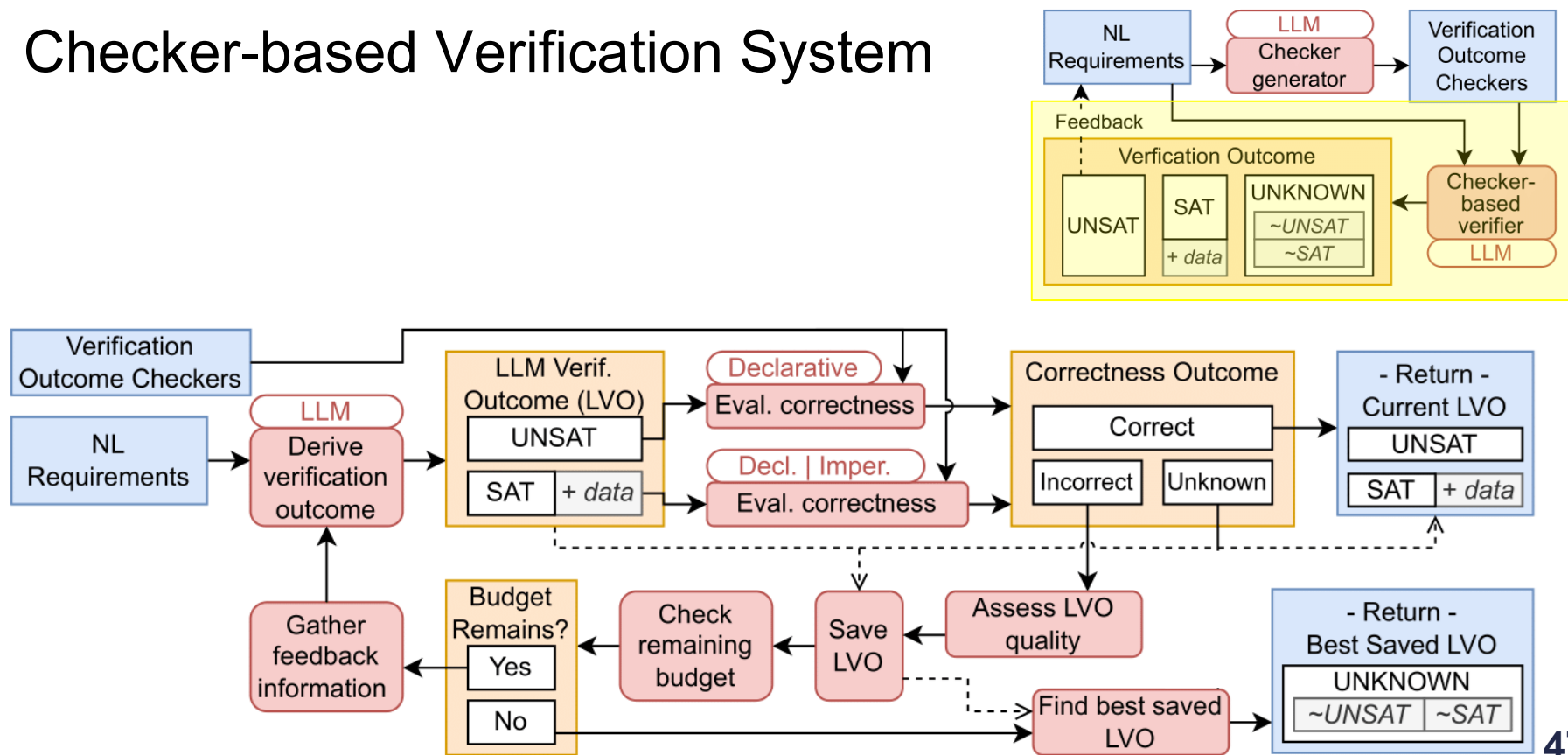
Checker-based Verification System



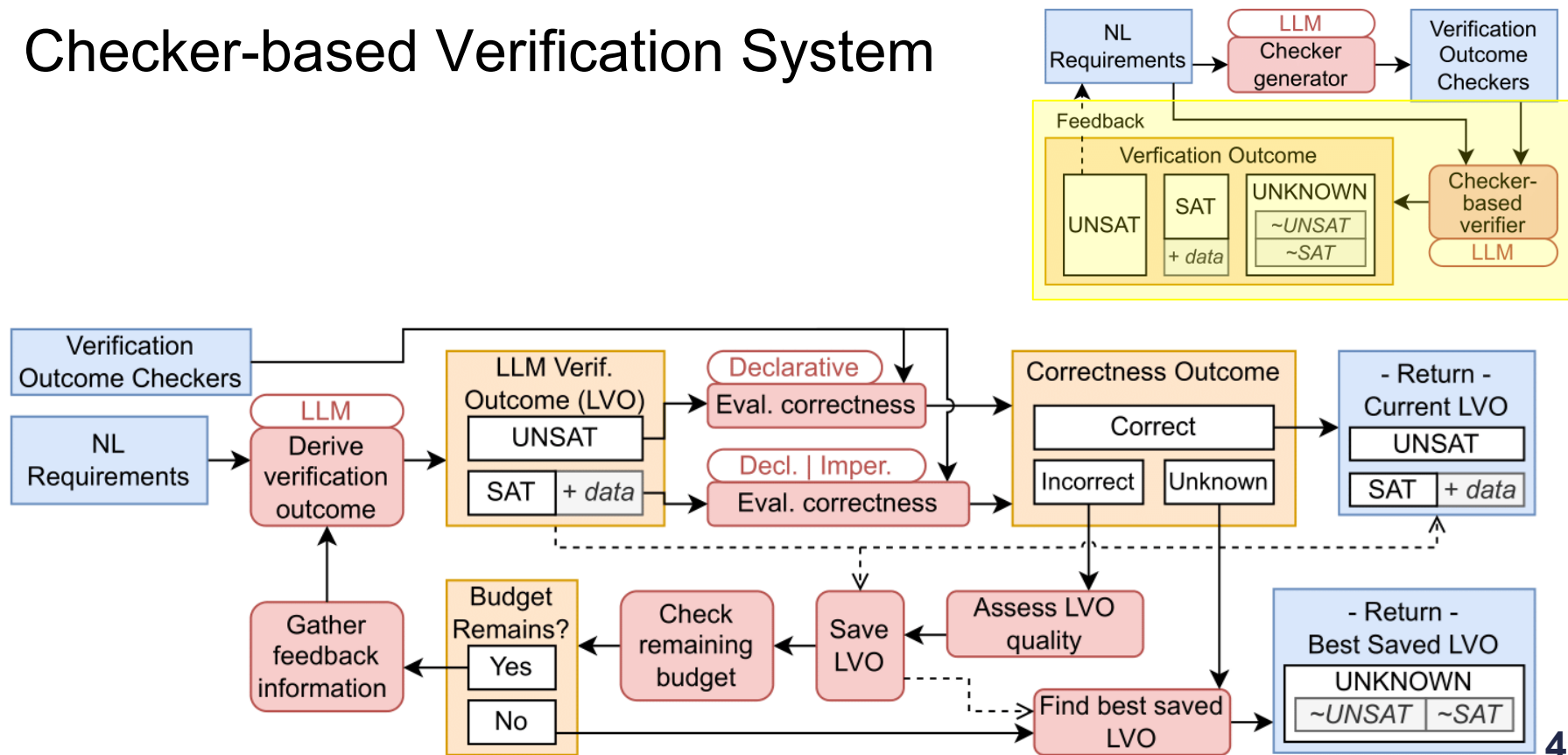
Checker-based Verification System



Checker-based Verification System

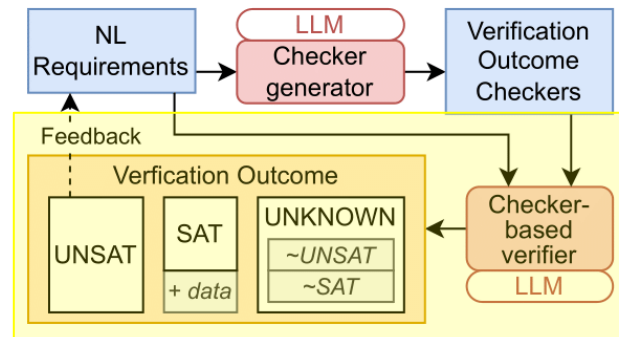


Checker-based Verification System



Failure Modes

+V: retry without any feedback,

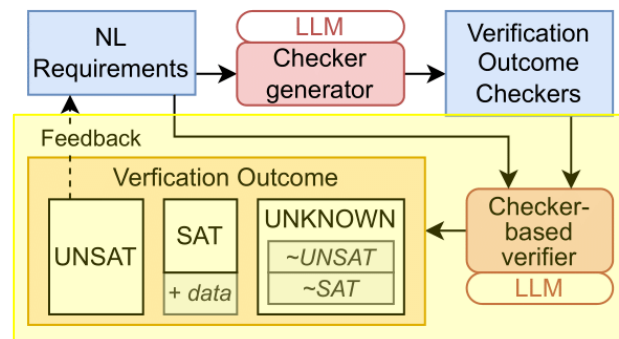


{...original LVO generation prompt}

Failure Modes

+V: retry without any feedback,

+VF: retry with past LVOs as counterexamples,



{...original LVO generation prompt}

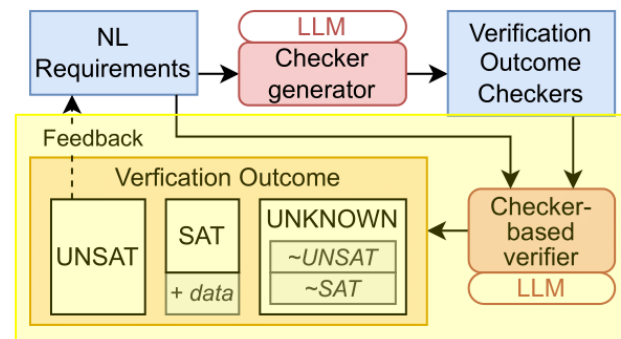
Here's the list of incorrect LVOs from previous iterations: {...}

Failure Modes

+V: retry without any feedback,

+VF: retry with past LVOs as counterexamples,

+VFE: retry with past LVOs, specifying the requirements failed.



{...original LVO generation prompt}

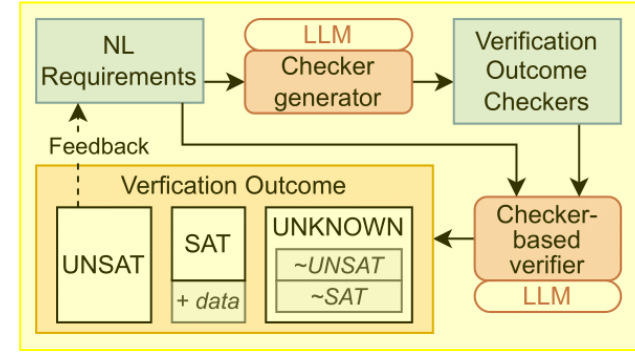
Here's the list of incorrect LVOs from previous iterations:

{LVO A} failed to satisfy {...requirement},

{LVO B} failed to satisfy {...requirement},

{...}

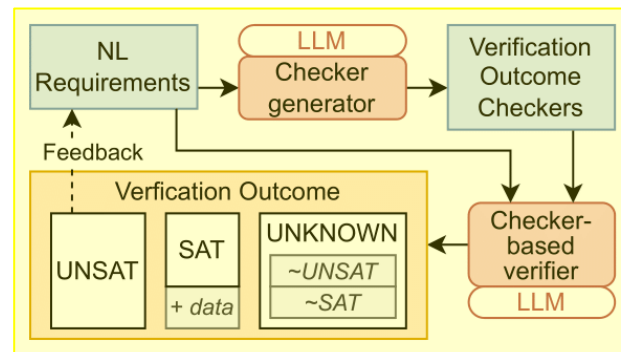
LVO Soundness Metrics



LVO Soundness Metrics

Generation Success Rate (GSR)

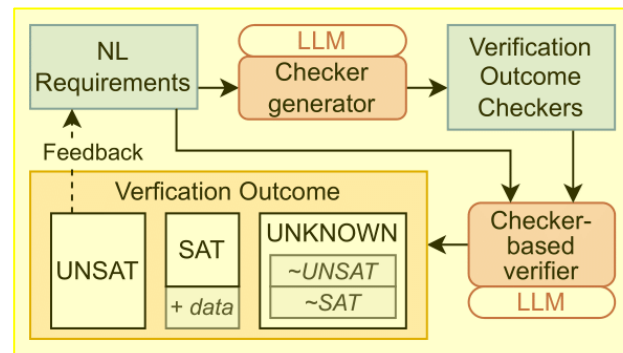
Proportion of LLM generations that yield a sensible, error-free LVO.



LVO Soundness Metrics

Generation Success Rate (GSR)

Proportion of LLM generations that yield a sensible, error-free LVO.



True/False Positive

A set of requirements is determined to be **SAT**, while it is in fact **SAT/UNSAT**.

True/False Negative

A set of requirements is determined to be **UNSAT**, while it is in fact **UNSAT/SAT**.

LVO Soundness Metrics

Generation Success Rate (GSR)

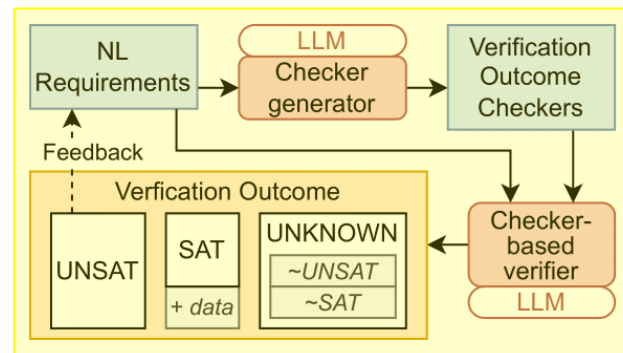
Proportion of LLM generations that yield a sensible, error-free LVO.

True/False Positive

A set of requirements is determined to be **SAT**, while it is in fact **SAT/UNSAT**.

True/False Negative

A set of requirements is determined to be **UNSAT**, while it is in fact **UNSAT/SAT**.

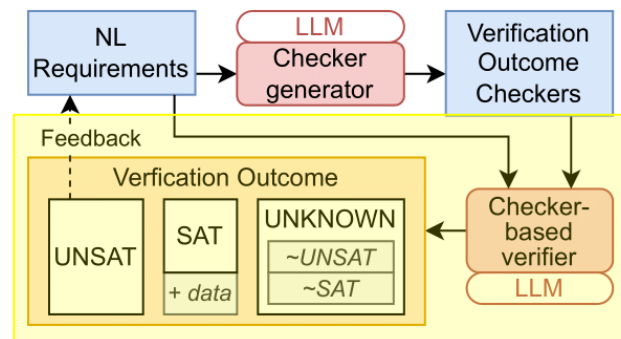


$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}$$

Soundness of LVOs



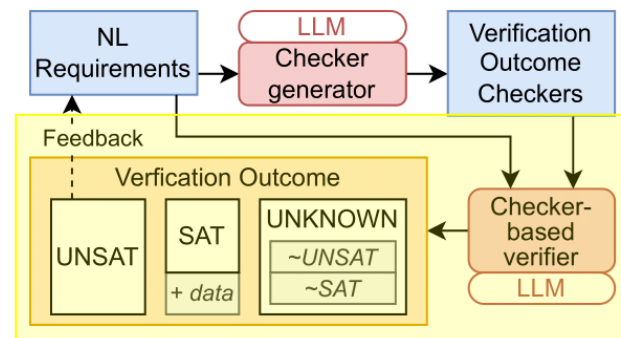
Methods	GPT-4o-mini		GPT-4o		DeepSeek-V3		Llama3.1-8b	
	GSR	F1	GSR	F1	GSR	F1	GSR	F1
Direct	66.43	59.83	78.45	63.64	72.08	58.49	25.80	34.29
+V	77.39	67.21	<u>92.23</u>	65.91	<u>91.87</u>	71.43	53.36	65.11
+VF	<u>78.09</u>	81.82	90.11	91.74	90.81	90.76	51.94	85.78
+VFE	77.74	<u>82.59</u>	<u>92.23</u>	<u>94.64</u>	<u>91.87</u>	<u>93.04</u>	<u>57.24</u>	<u>87.10</u>

* using ground truth checkers

Soundness of LVOs

Incorporating verification improves LVO quality,

- The quality of LVOs generated using verification significantly outperforms those without verification across all metrics and configurations.



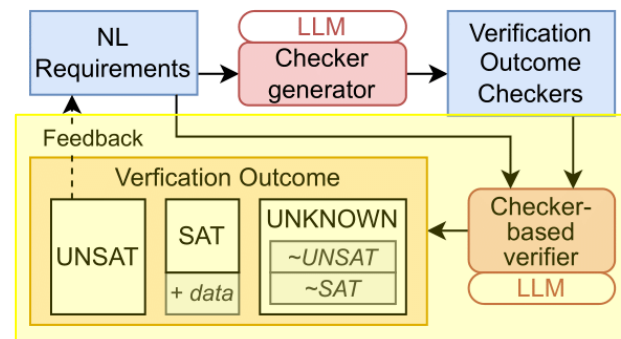
Methods	GPT-4o-mini		GPT-4o		DeepSeek-V3		Llama3.1-8b	
	GSR	F1	GSR	F1	GSR	F1	GSR	F1
Direct	66.43	59.83	78.45	63.64	72.08	58.49	25.80	34.29
+V	77.39	67.21	<u>92.23</u>	65.91	<u>91.87</u>	71.43	53.36	65.11
+VF	<u>78.09</u>	81.82	90.11	91.74	90.81	90.76	51.94	85.78
+VFE	77.74	<u>82.59</u>	<u>92.23</u>	<u>94.64</u>	<u>91.87</u>	<u>93.04</u>	<u>57.24</u>	<u>87.10</u>

* using ground truth checkers

Soundness of LVOs

Incorporating feedback improves LVO quality,

- **+VFE** achieves the best results, providing the highest GSR in three out of four LLMs and the best F1-scores across all evaluated LLMs.

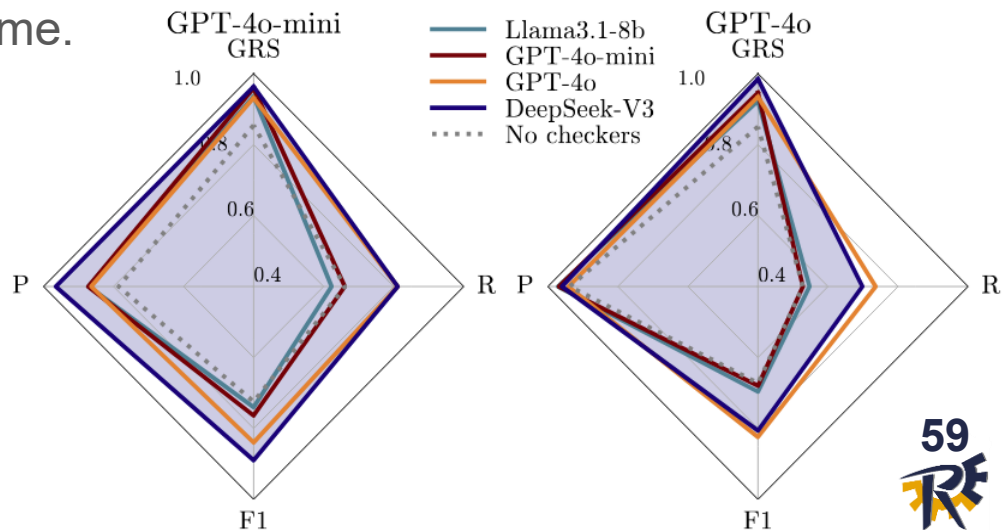
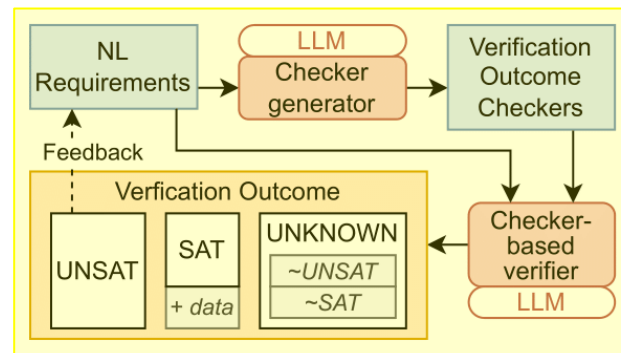


Methods	GPT-4o-mini		GPT-4o		DeepSeek-V3		Llama3.1-8b	
	GSR	F1	GSR	F1	GSR	F1	GSR	F1
Direct	66.43	59.83	78.45	63.64	72.08	58.49	25.80	34.29
+V	77.39	67.21	<u>92.23</u>	65.91	<u>91.87</u>	71.43	53.36	65.11
+VF	<u>78.09</u>	81.82	90.11	91.74	90.81	90.76	51.94	85.78
+VFE	<u>77.74</u>	<u>82.59</u>	<u>92.23</u>	<u>94.64</u>	<u>91.87</u>	<u>93.04</u>	<u>57.24</u>	<u>87.10</u>

* using ground truth checkers

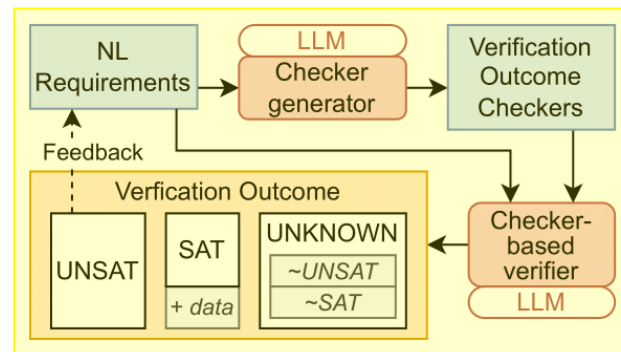
End-to-End Soundness of LVOs

- Higher checker quality consistently yields higher LVO quality,
 - Verifiers using the highest-quality checkers generated by GPT-4o and DeepSeek-V3 generally yield the best outcome.

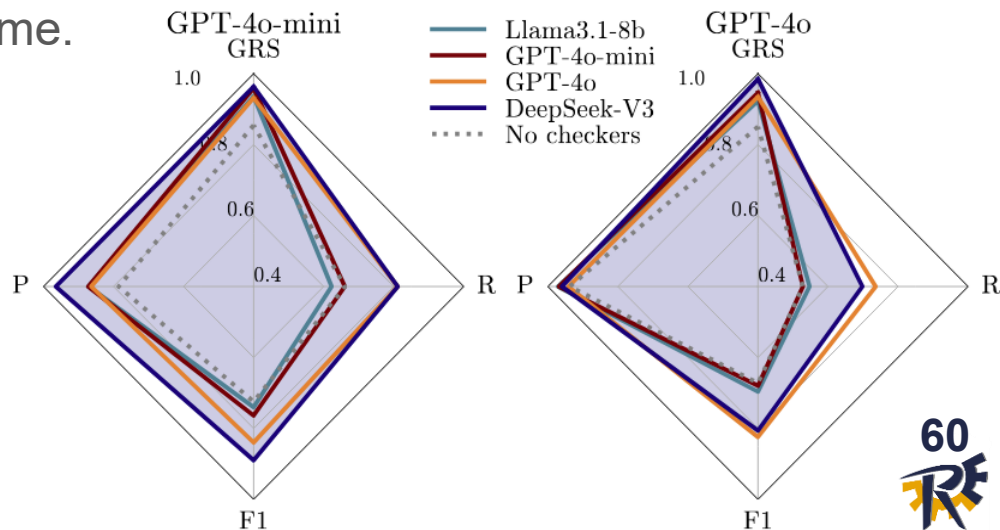
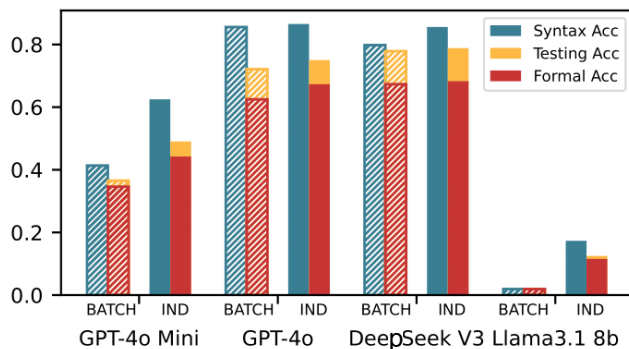


End-to-End Soundness of LVOs

- Higher checker quality consistently yields higher LVO quality,
 - Verifiers using the highest-quality checkers generated by GPT-4o and DeepSeek-V3 generally yield the best outcome.



Soundness of Declarative Checkers



Key Takeaways

Key Takeaways



We propose an approach that leverages LLMs for the verification of natural language requirements for satisfiability,

Key Takeaways



We propose an approach that leverages LLMs for the verification of natural language requirements for satisfiability,



We experimentally show that requirements verification benefits from integrating declarative and imperative checkers with detailed feedback,

Key Takeaways



We propose an approach that leverages LLMs for the verification of natural language requirements for satisfiability,



We experimentally show that requirements verification benefits from integrating declarative and imperative checkers with detailed feedback,



How can we improve the accuracy of declarative (SMT) checker generation?

Key Takeaways



We propose an approach that leverages LLMs for the verification of natural language requirements for satisfiability,



We experimentally show that requirements verification benefits from integrating declarative and imperative checkers with detailed feedback,



How can we improve the accuracy of declarative (SMT) checker generation?



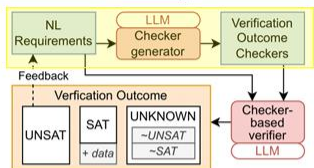
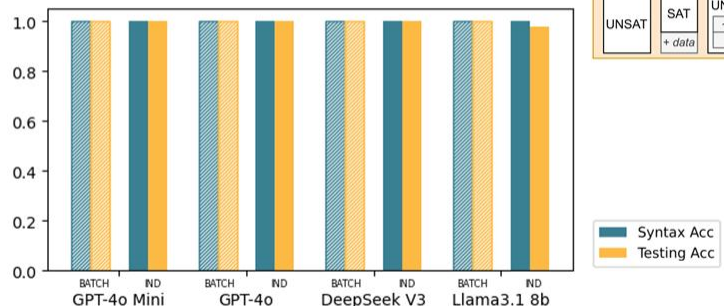
Apart from string requirements, what are other textual formalism we may explore using this approach?

Example of Requirements Verification

- R1.** The system shall require an email address to contain exactly one “@” character.
- R2.** The system shall require an email address not to end with a “.” character.
- R3.** The system shall require an email to contain at least one “.” character after the “@” character.

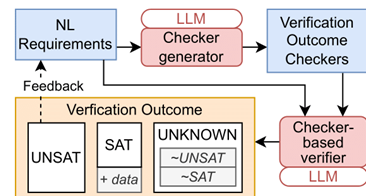
Requirement Set	Behavior	Satisfiable?	Example Data
$R1 \wedge R2 \wedge R3$	Store email	Yes	“ab@cd.com”
$\neg R1 \wedge R2 \wedge R3$	Show error	Yes	“ab@@cd.com”
$R1 \wedge \neg R2 \wedge \neg R3$	Show error	No	-

Soundness of Imperative (Python) Checkers



Our Proposition

- Using large language models (LLMs):
 - Generate Requirement Verification Checkers,
 - Generate the LLM-based Verification Outcome (LVO),
- Use checkers to verify the soundness of the LVO and retry if necessary.



End-to-End Soundness of LVOs

- Higher checker quality consistently yields higher LVO quality,
 - Verifiers using the highest-quality checkers generated by GPT-4o and DeepSeek-V3 generally yield the best outcome.

Soundness of Declarative Checkers

